

4

# Score-Based Generative Modeling Using Stochastic Differential Equations

By

Navya-EE23B145

Nandini-EP23B012

- **What is the overall overview of this project?**

This project explores the evolution of modern image generation models and focuses on how score-based generative models use stochastic differential equations (SDEs) to generate highly realistic images

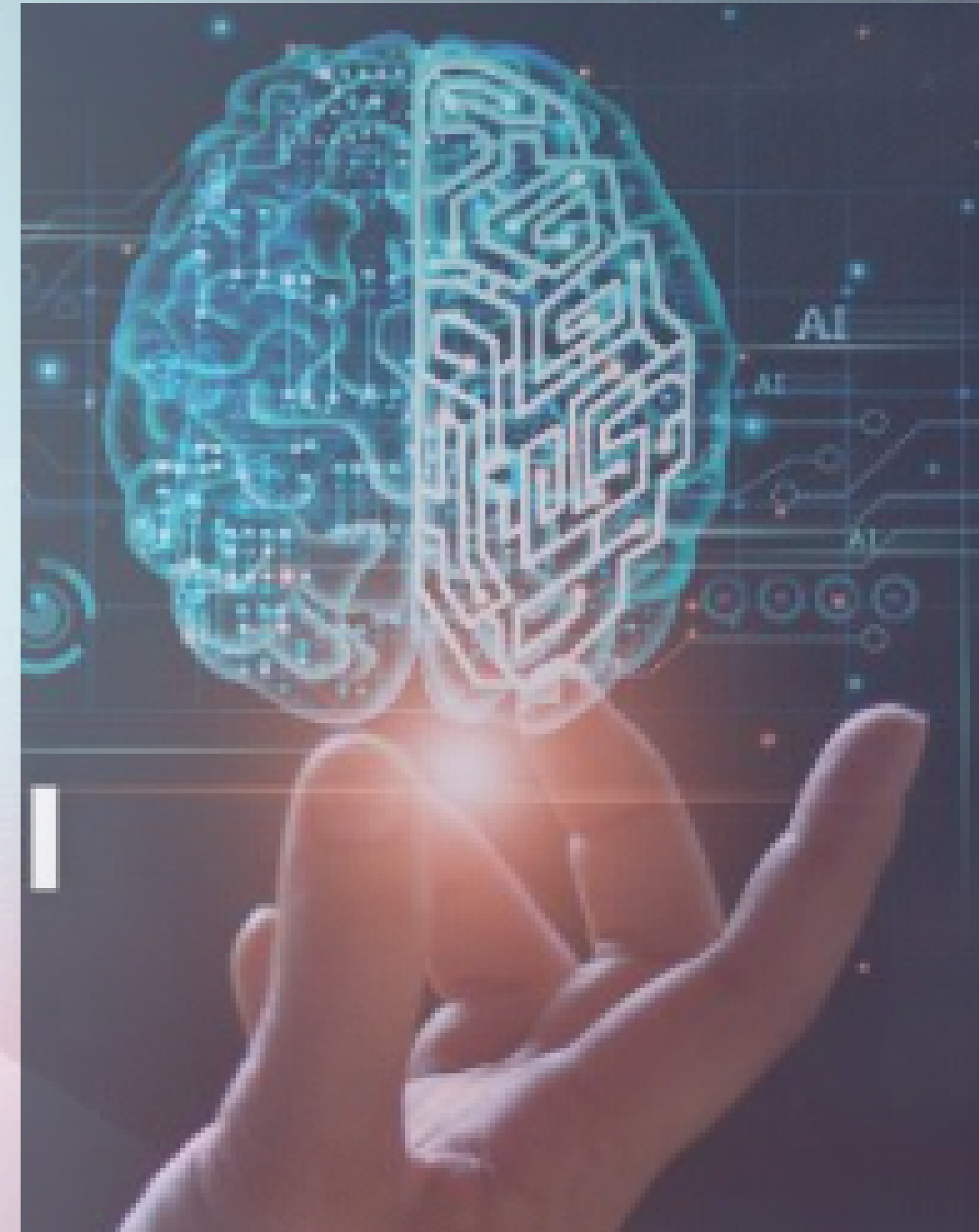
- **Evolution of Image Generation Models**

- 1.Traditional Approaches**

- Statistical Models
    - Autoencoders

- 2.Modern Deep Learning Approaches**

- Variational Autoencoders (VAEs)
    - Generative Adversarial Networks (GANs)
    - Diffusion Models
    - Score-Based Generative Models using SDEs



# Statistical Models

The data points in a training dataset come from an underlying data distribution. To estimate this distribution, we build a model that learns a parameterized probability distribution data. Once the distribution is learned:

- new images can be generated through sampling,
- probability evaluation helps determine realistic outputs.

## Process:

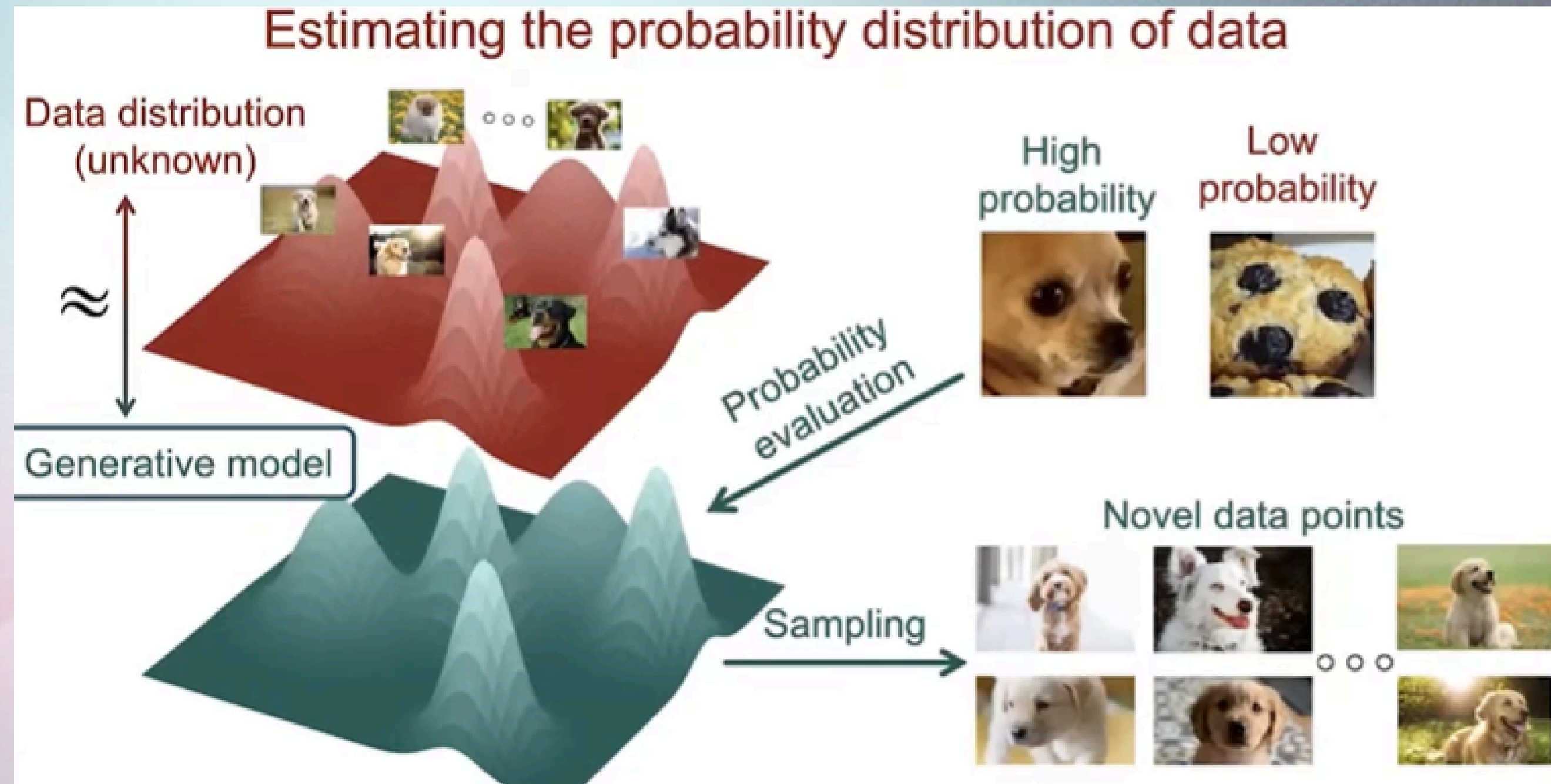
Training Data →

Learn Distribution →

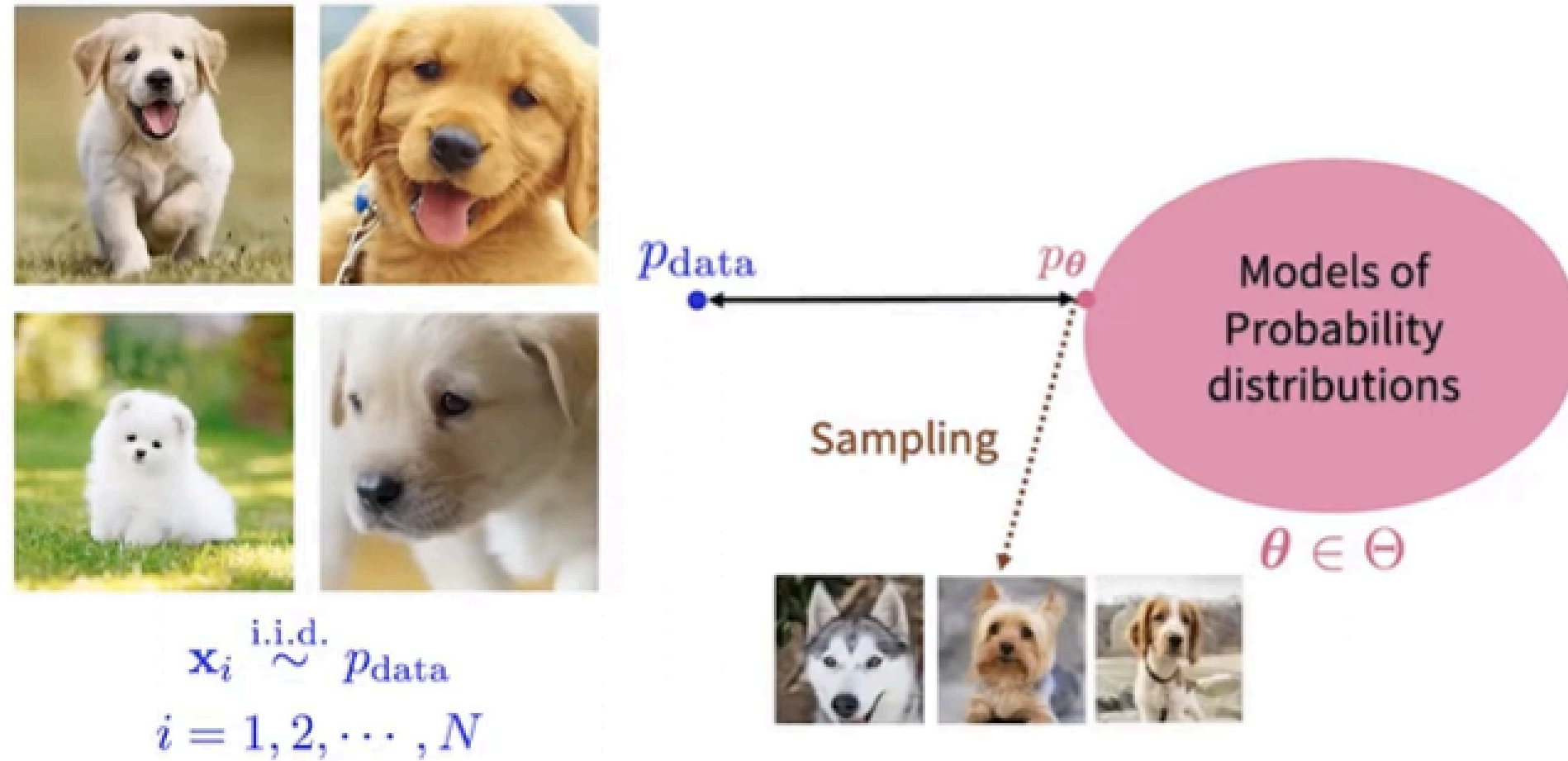
Sampling →

Generate New Images

These models are called **generative models** because they can **generate new data samples**.



## Training generative models



The model tries to estimate the true data distribution:

$$P_{data}(x)$$

using a parameterized model:

$$P_{\theta}(x)$$

Where:

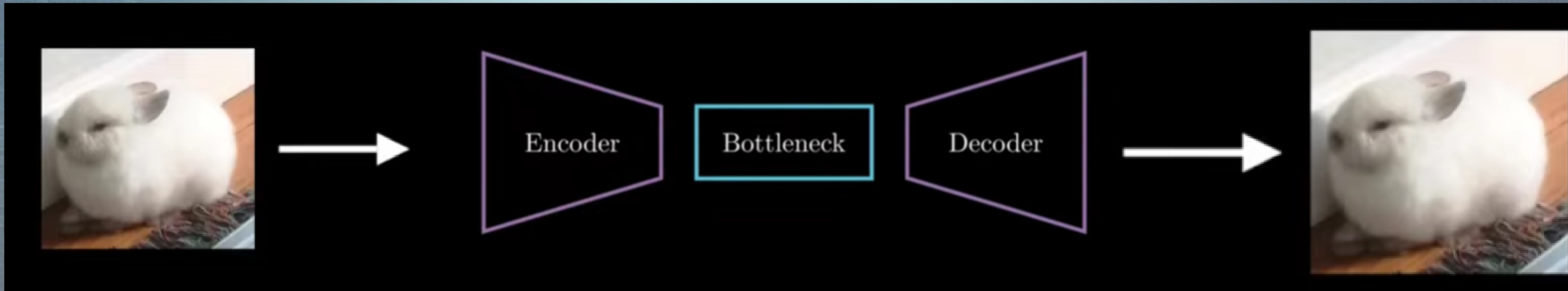
$\theta$  = learnable parameters of the neural network.

## Key challenge for building complex generative models:

- Data Distribution is extremely complex for higher dimensional data.
- How to build a complex model to fit the data distribution?

**“Due to These Challenges, Autoencoders Were Introduced”**

# Auto Encoders: Learning Compressed Representations of Data



An Autoencoder is a neural network that learns to compress complex and high dimensional input data into compact representation & reconstruct the original data from the compressed form. Our aim is to minimize this reconstruction loss.

Input Image → Encoder → Latent Space → Decoder → Reconstructed Image

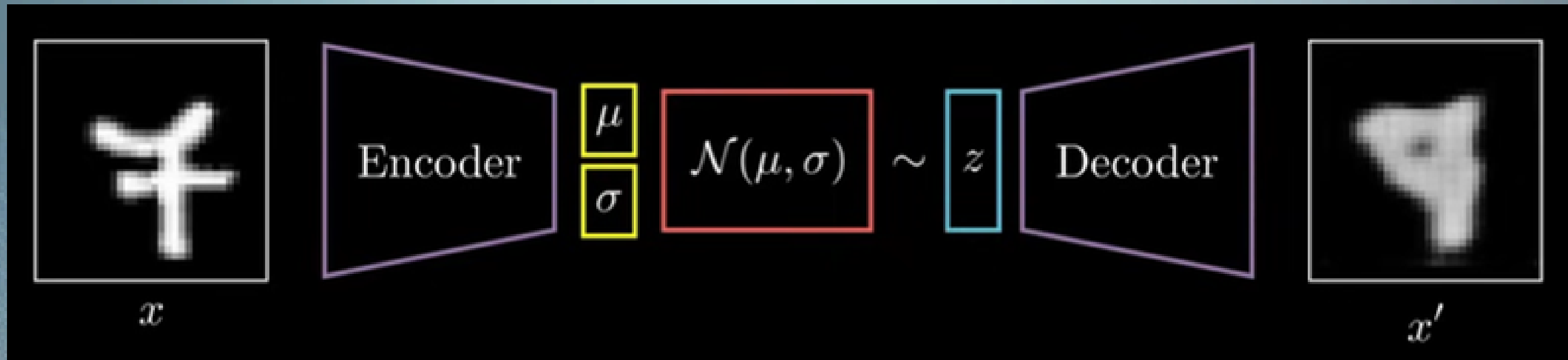
## Applications

- Image compression
- Feature extraction
- Anomaly detection

## Limitations

- Cannot generate highly realistic images
- Reconstruction quality depends on latent dimension
- Simple autoencoders are not strong generative models

# Variational Autoencoders: Probabilistic Generative Models

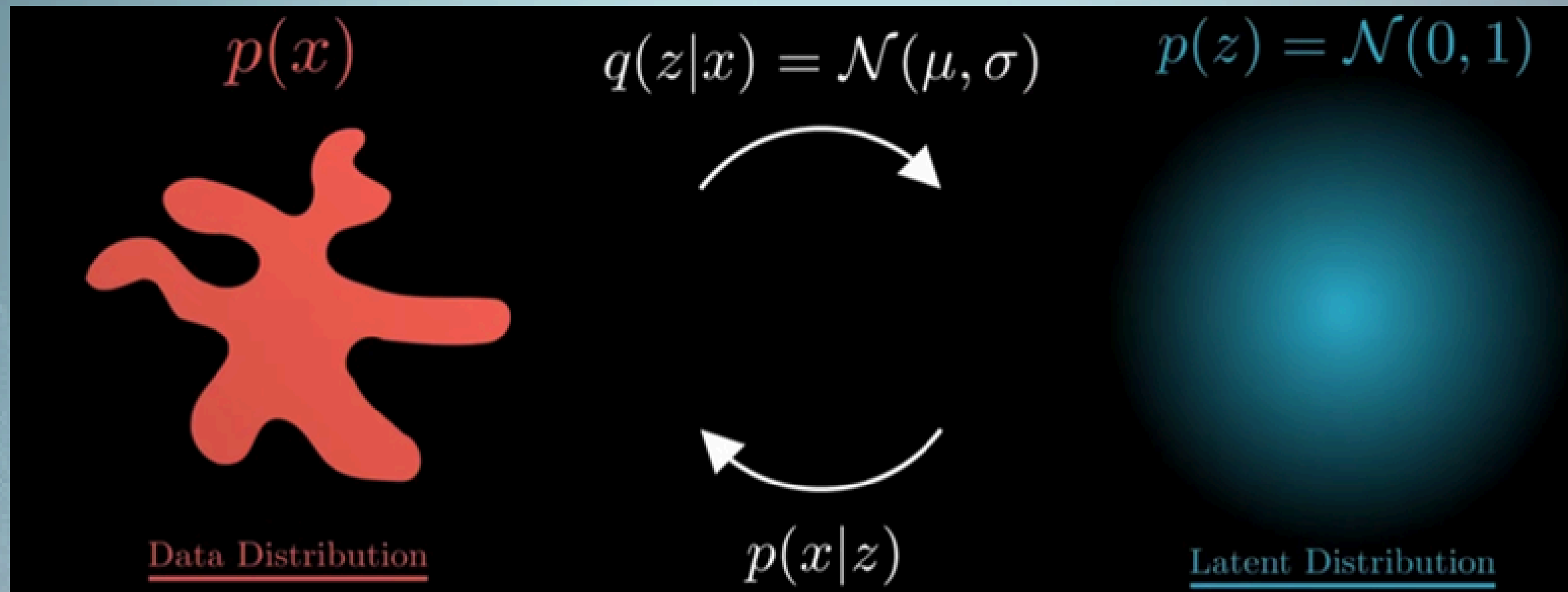


A Variational Autoencoder (VAE) is a generative deep learning model that:

- learns compressed latent representations
- models probability distributions
- VAEs learn a distribution instead of fixed latent points.

Input Image  $\rightarrow$  Encoder  $\rightarrow$  Mean & Variance  $\rightarrow$  Latent Space Sampling  $\rightarrow$  Decoder  $\rightarrow$  Generated/Reconstructed Image

- The encoder learns two parameters:
- Mean ( $\mu$ )
- Standard deviation ( $\sigma$ ) - which define the latent distribution. A latent vector is sampled from this distribution and passed to the decoder for reconstruction.



- Training uses two losses: The Reconstruction Loss and The KL Divergence Loss
- The reparameterization trick enables backpropagation during random sampling.

### How do we train this ?

$$\mathcal{L}(x) = \mathbb{E}_{q(z|x)} [\log p(x|z)] - \text{KL}(q(z|x) \parallel p(z))$$

#### Advantages

- Smooth latent space
- Better representation learning

#### Limitations

- Generated images may appear blurry
- Lower sharpness compared to GANs

# Generative Adversarial Networks (GANs)

GANs consist of two neural networks:

- Generator
- Discriminator

**GAN Workflow:** Random Noise →

Generator → Fake Image →

Discriminator → Real/Fake Prediction

- Generator creates fake images from random noise & Discriminator

checks whether an image is real or generated.

- Both networks are trained simultaneously in a competitive process called adversarial training.

## Advantages of GANs

- Generate sharp and realistic images
- Fast image generation after training



## Limitations of GANs

- Training can be unstable
- Difficult to balance generator and discriminator training

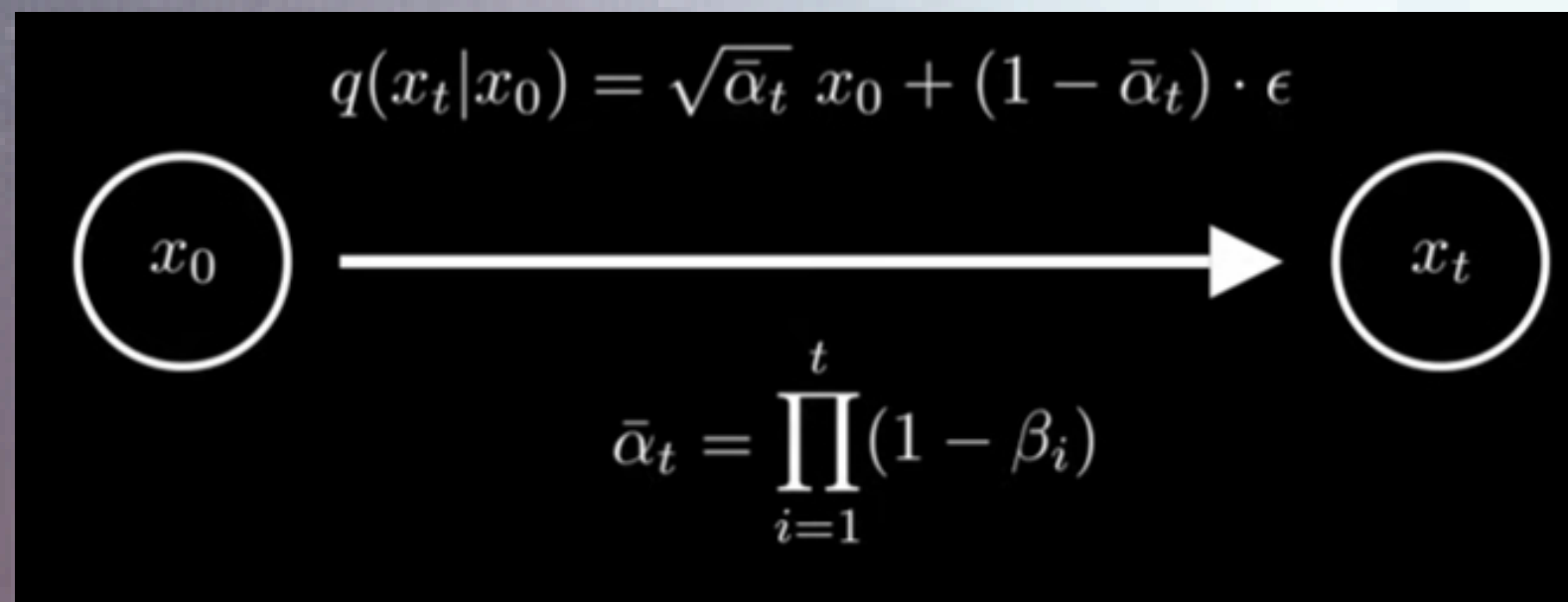
# Introduction to Diffusion Models

- What are Diffusion Models?

Diffusion models are generative models that learn to create realistic images by reversing a gradual noising process.

Diffusion Process : forward - Real Image  $\rightarrow$  Add Noise Gradually  $\rightarrow$  Pure Noise  
backward process-Pure Noise  $\rightarrow$  Remove Noise Step-by-Step  $\rightarrow$  Generated Image

## Forward Diffusion Process



## Backward Diffusion process

$$p_{\theta}(x_{0:T}) = p_{\theta}(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t)$$

Complete reverse process

The goal is to maximize the likelihood of generating real images. Instead of maximizing likelihood directly, training minimizes:

$$\text{Negative Log-Likelihood} \\ -\log p_{\theta}(x_0)$$

## Evidence Lower Bound (ELBO)

$$-\mathbb{E}_{q(x_{1:T}|x_0)} \left[ \log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T} | x_0)} \right]$$

The first term is ignored because it is constant and does not depend on the trainable  $\theta$ , while the last term is ignored because its contribution becomes very small during optimization.

$$-\mathbb{E}_q \left[ D_{\text{KL}}(q(x_T | x_0) || p(x_T)) + \sum_{t>1} D_{\text{KL}}(q(x_{t-1} | x_t, x_0) || p_{\theta}(x_{t-1} | x_t)) - \log p_{\theta}(x_0 | x_1) \right]$$

$$-\mathbb{E}_q \left[ \sum_{t>1} D_{\text{KL}}(q(x_{t-1} | x_t, x_0) || p_{\theta}(x_{t-1} | x_t)) \right]$$

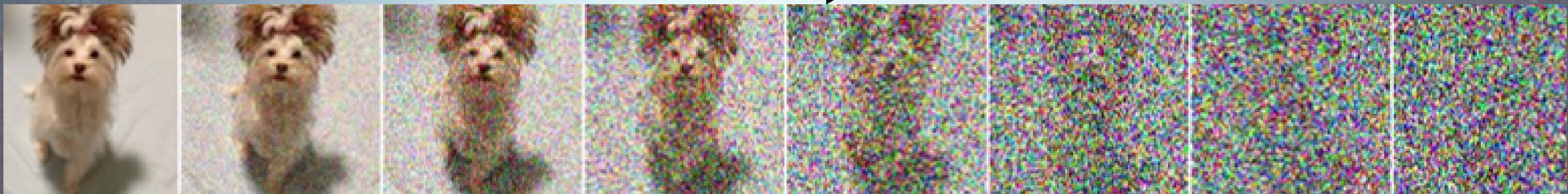
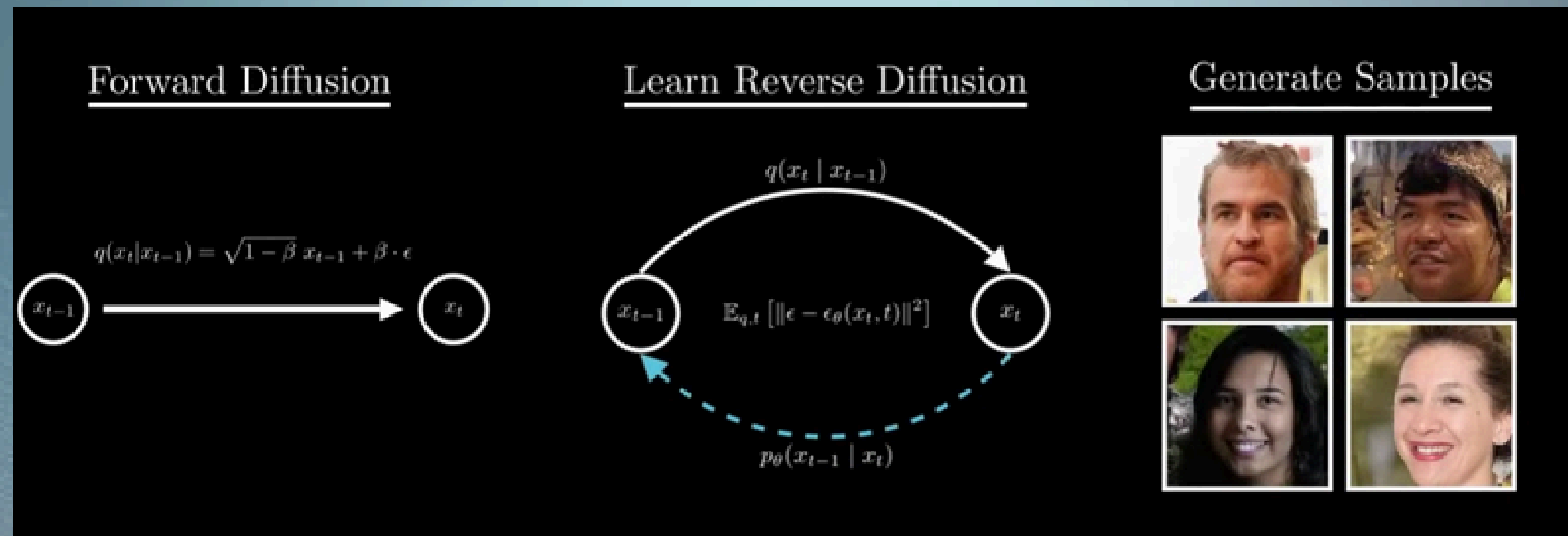
↓

$$\mathbb{E}_q \left[ \sum_{t>1} \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t - \mu_{\theta}(x_t, t)\|^2 \right]$$

## Final Loss Function:

$$\mathcal{L} = \mathbb{E}_q \left[ \sum_{t>1} \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2 \right]$$

The diffusion model training process starts from minimizing the negative log-likelihood, derives the ELBO objective, rewrites and simplifies it using KL divergences and Gaussian assumptions, and finally uses reparameterization trick to convert the entire optimization into a simple neural network noise prediction loss.



To overcome the limitations of traditional diffusion models and achieve a more flexible, mathematically continuous framework, researchers introduced Score-Based Generative Models using stochastic differential equations.

# Score Based Generative Modelling using Stochastic Differential Equations:

- Model diffusion continuously over time
- Provide strong probabilistic foundation
- Improve stability and image quality

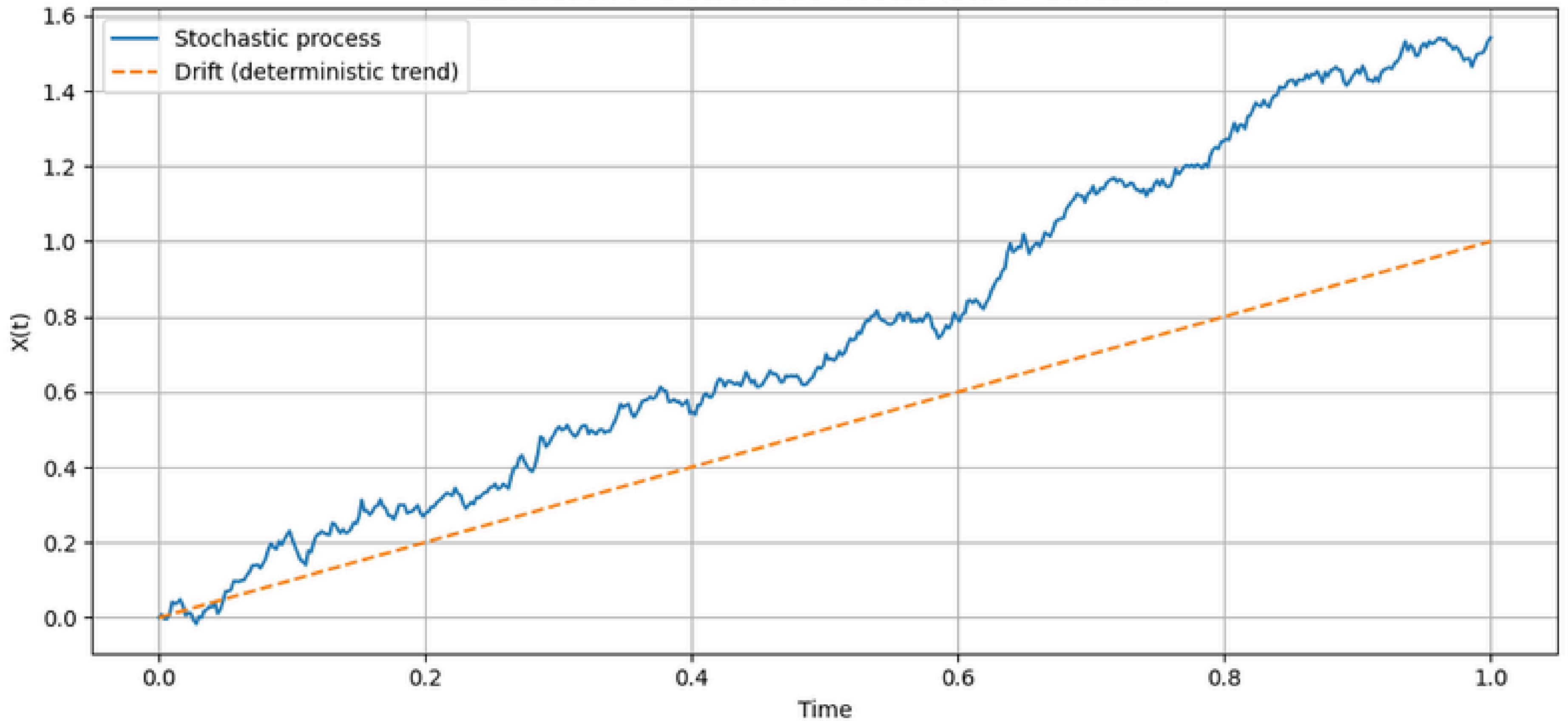
**General SDE :**

$$dx = f(x, t)dt + g(t)dW$$

**Components:**

- $f(x, t)$  → Drift term (deterministic evolution)
- $g(t)$  → Diffusion coefficient (controls noise strength)
- $dW$  → Brownian motion / Gaussian random noise

Simulation of a Stochastic Process (Drift + Diffusion)

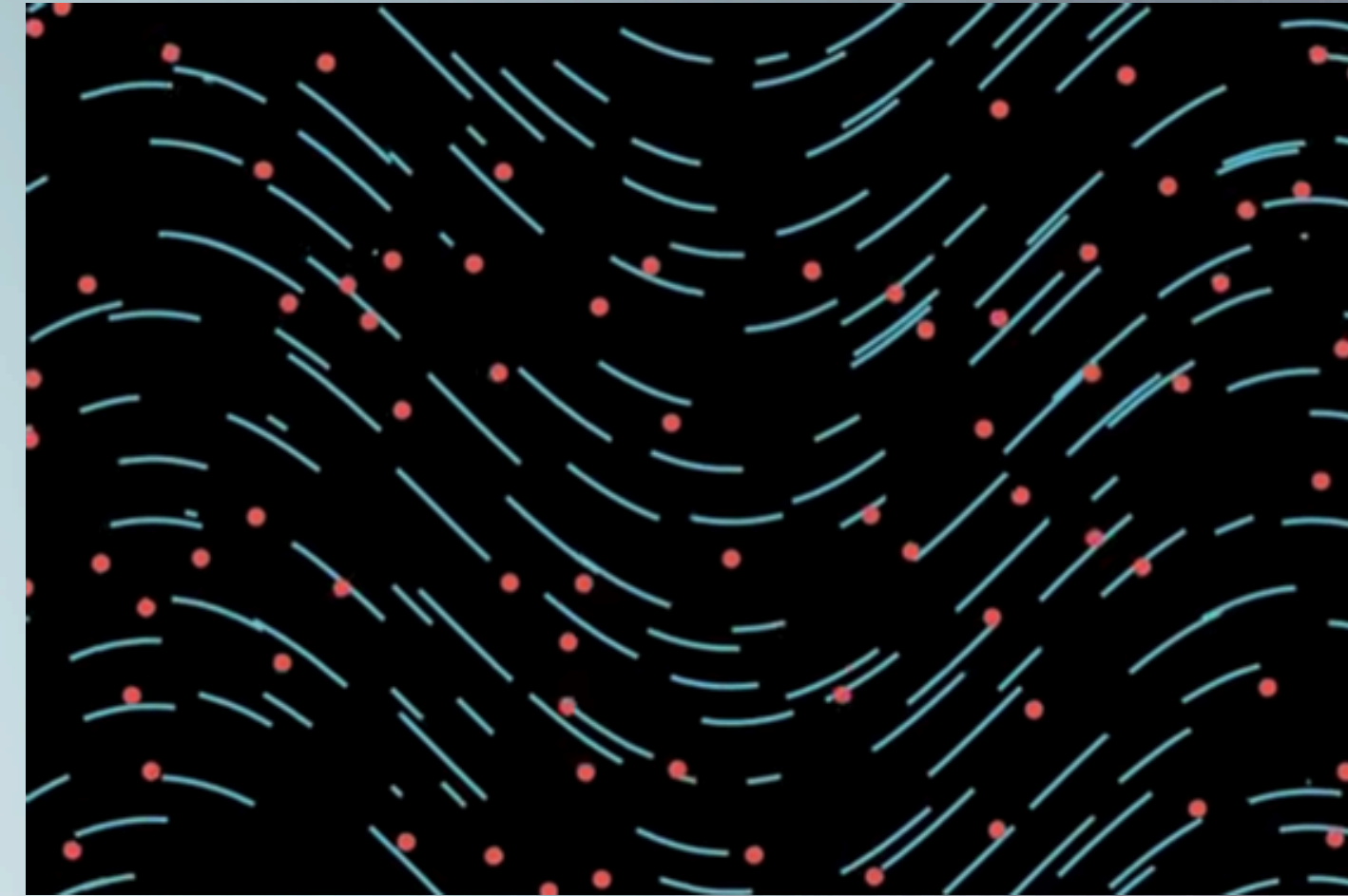


# DDPM Markov Chain

$$x_i = \sqrt{1 - \beta_i} x_{i-1} + \sqrt{\beta_i} \epsilon_{i-1} \quad \tilde{\beta}_i = N \beta_i$$

$$x_i = \sqrt{1 - \frac{\tilde{\beta}_i}{N}} x_{i-1} + \sqrt{\frac{\tilde{\beta}_i}{N}} \epsilon_{i-1} \quad \Delta t = \frac{1}{N}$$

$$x_i = x(t + \Delta t) \quad \tilde{\beta}_i = \beta(t + \Delta t)$$



$$x(t + \Delta t) = \sqrt{1 - \beta(t + \Delta t) \Delta t} x(t) + \sqrt{\beta(t + \Delta t) \Delta t} \epsilon(t)$$

$$x(t + \Delta t) \approx x(t) - \frac{1}{2} \beta(t + \Delta t) x(t) \Delta t + \sqrt{\beta(t + \Delta t) \Delta t} \epsilon(t)$$

$$x(t + \Delta t) - x(t) \approx -\frac{1}{2} \beta(t) x(t) \Delta t + \sqrt{\beta(t) \Delta t} \epsilon(t) \quad \xrightarrow{\lim \Delta t \rightarrow 0} \quad dx \approx -\frac{1}{2} \beta(t) x(t) dt + \sqrt{\beta(t)} \sqrt{dt} \epsilon(t)$$

Drift term : gradually reduces the original signal/information.drift pulls toward origin

Diffusion term:gradually adds random Gaussian noise to the data.spreads them around

## Forward SDE:

- Gradually adds Gaussian noise to data
- Destroys image structure over time

### Process:

Image → Slight Noise → Heavy Noise → Pure Gaussian Noise

- Transform complex data distribution into simple noise distribution

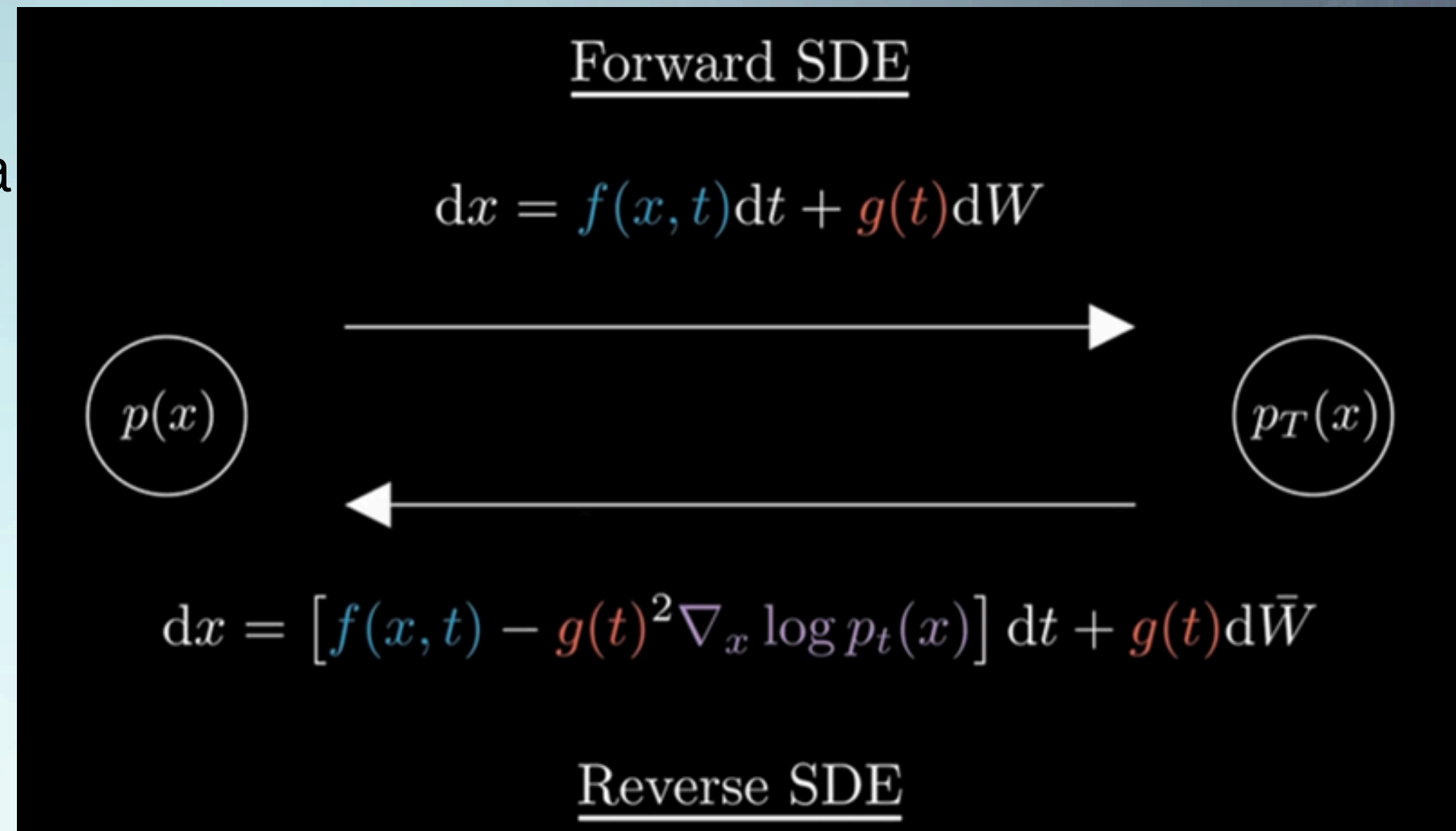
## Reverse SDE:

- Reverse process removes noise step-by-step
- Noise → Cleaner Image → Realistic Image
- Uses learned **score function**:

$$s_{\theta}(x, t) = \nabla_x \log p_t(x)$$

acts as a denoising direction. to guide samples toward realistic data regions.

“Depending on the forward diffusion process, score-based models use either VP-SDE or VE-SDE.



# Variance Exploding SDE (VE SDE):

In VE SDE:

- noise variance continuously increases over time.
- the data spreads outward heavily.

**Forward VE SDE:**  $dx = \sqrt{\frac{d\sigma^2(t)}{dt}} dW$

**Reverse VE SDE:**

$$dx = -g(t)^2 s_{\theta}(x, t) dt + g(t) d\bar{W}$$

## Advantages of VE SDE

- Strong Exploration
- Stable Score Estimation
- Large noise smooths distributions.
- Good for Continuous Modeling
- Works naturally with score matching.

**Observations in Forward VE SDE:**

- There is no drift term.
- only diffusion/noise term.
- This means, data is continuously perturbed by increasing noise.

## Limitations of VE SDE

- Excessive Noise
- Can make reconstruction harder.
- Sampling Cost
- May require many denoising steps.

# Variance Preserving SDE (VP SDE):

VP SDE balances:

- signal decay & noise injection.

Overall variance remains approximately constant.

**Forward VP SDE:**  $dx = -\frac{1}{2}\beta(t)x dt + \sqrt{\beta(t)} dW$

*Two important terms:*

- Drift Term :  $-\frac{1}{2}\beta(t)x dt$  -This gradually shrinks the signal.
- Diffusion Term :  $\sqrt{\beta(t)} dW$  -This adds Gaussian noise.

**Reverse VP SDE:**  $dx = \left[ -\frac{1}{2}\beta(t)x - \beta(t)s_{\theta}(x, t) \right] dt + \sqrt{\beta(t)} d\bar{W}$

## Advantages of VP SDE:

- Stable Training
- High Image Quality
- Widely used in modern diffusion systems.
- Controlled Noise
- Better balance between signal and randomness.

## Limitations of VP SDE:

- Slower Exploration
- Less aggressive than VE.
- Sampling Complexity
- computationally expensive.

# Sub-Variance Preserving SDE (Sub-VP SDE):

Sub-VP modifies VP SDE to:

- inject less noise,
- preserve more signal information.

It combines advantages of VE SDE & VP SDE.

**Forward Sub-VP SDE:**  $dx = -\frac{1}{2}\beta(t)x dt + \sqrt{\beta(t) \left(1 - e^{-2\int_0^t \beta(s) ds}\right)} dW$

**Reverse Sub-VP SDE:**  $dx = \left[-\frac{1}{2}\beta(t)x - g(t)^2 s_\theta(x, t)\right] dt + g(t) d\bar{W}_t g(t) = \sqrt{\beta(t) \left(1 - e^{-2\int_0^t \beta(s) ds}\right)}$

Compared to VP:

- diffusion/noise term is reduced.

This means more structure is preserved & better likelihood estimation

## Advantages of Sub-VP:

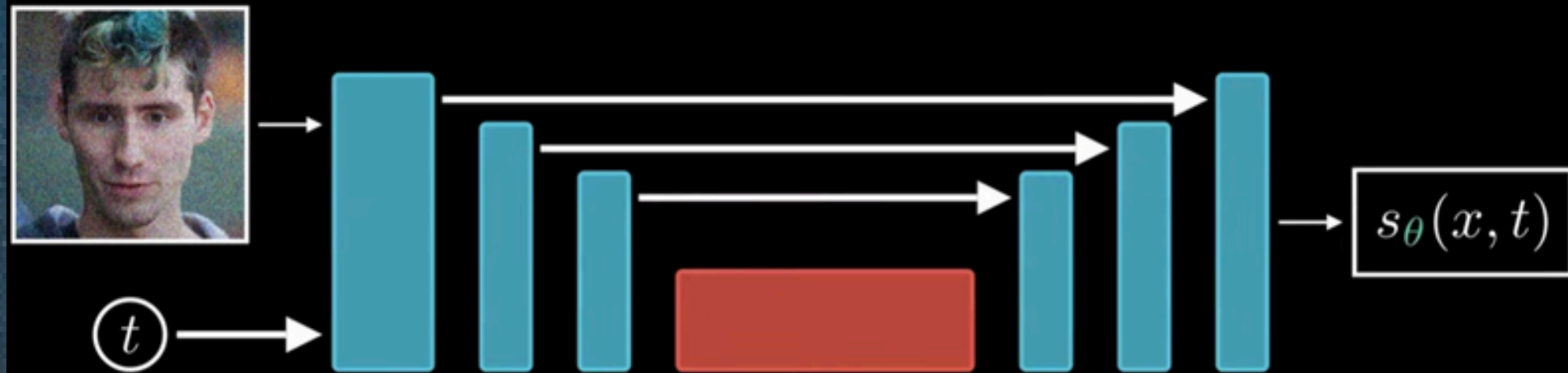
- Better Likelihood Performance
- Improves probabilistic estimation.
- More Signal Retention
- Easier reconstruction.
- Efficient Sampling, Can improve generation quality.

## Limitations of Sub-VP:

- More Complex
- Mathematically more involved.
- Less Common
- Not as widely adopted as VP.

# U-net architecture

The network architecture



## Why U-net:

- The U-Net is trained using the denoising score matching loss to learn the amount of noise present in a noisy image at a particular timestep and guide image toward clean output during reverse diffusion.

## How do we train?

### Denoising Score Matching loss

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0 \sim p(x), t} \left[ \left\| s_{\theta}(x, t) + \frac{\epsilon}{\beta_t} \right\|^2 \right]$$

This equation is used to train U-Net in diffusion models. A noisy image and its timestep are given to the U-Net, which predicts the noise present in the image. The loss function compares the predicted noise with the actual added noise and updates the network to improve denoising.

- Diffusion happens continuously, but computers cannot process continuous changes directly. So we use **Euler-Maruyama discretization** to divide the diffusion process into many small steps, making it possible to gradually add or remove noise during training and generation.

## Euler-Maruyama discretization

$$dx = \left[ -\frac{1}{2}\beta(t)x - \beta(t)s_{\theta}(x, t) \right] dt + \sqrt{\beta(t)}d\bar{W}$$

The infinitesimal change  $dt$  is replaced by a small finite step  $\Delta t$ , and the difference between  $x_{t+1}$  and  $x_t$  represents the image update at each step.

The Brownian motion term is also discretized, producing the random noise term used to simulate diffusion numerically on a computer. results with this equation

$$x_{t+1} = x_t + \left[ -\frac{1}{2}\beta(t)x_t - \beta(t)s_{\theta}(x_t, t) \right] \Delta t + \sqrt{\beta(t)}\sqrt{\Delta t}\epsilon$$

now let's look at the difference between the Diffusion models and score based diffusion models

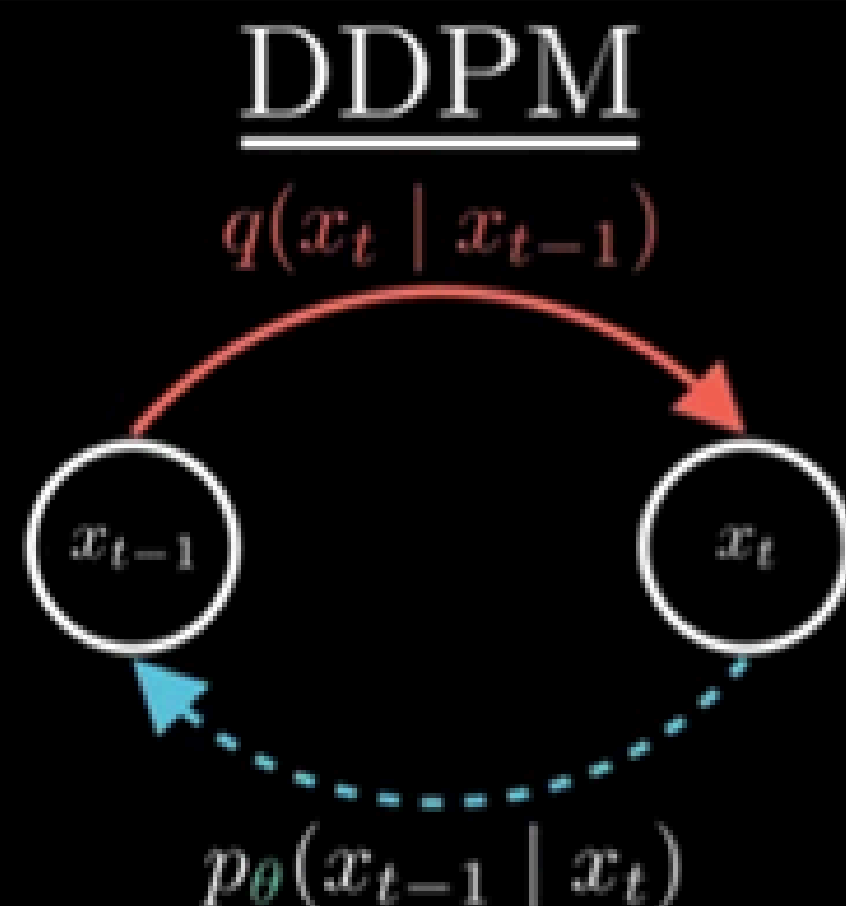
Noising Process

Reverse Process

Objective

Network Output

Training Loss



Match  $p_\theta(x_{t-1} | x_t)$  to  $q(x_{t-1} | x_t, x_0)$

$$\epsilon_\theta(x_t, t)$$

$$\mathbb{E}_{x_0, t, \epsilon} [\|\epsilon_\theta(x_t, t) - \epsilon\|^2]$$

Score-diffusion

$$dx = -\frac{1}{2}\beta(t)xdt + \sqrt{\beta(t)}dW$$



$$dx = \left[ -\frac{1}{2}\beta(t)x - \beta(t)\nabla_x \log p_t(x) \right] dt + \sqrt{\beta(t)}d\bar{W}$$

Learn the score  $-\nabla_x \log p_t(x)$

$$s_\theta(x_t, t)$$

$$\mathbb{E}_{x_0, t, \epsilon} [\|\sqrt{\beta(t)}s_\theta(x_t, t) + \epsilon\|^2]$$

the next important question is how we evaluate the quality of those generated images. Two widely used metrics for this are Fréchet Inception Distance (FID) and Inception Score (IS), which measure realism, quality, and diversity of generated samples.”

## Fréchet Inception Distance (FID)

- FID measures the distance between the feature distributions of real and generated images using embeddings extracted from a pretrained Inception-V3 (or ResNet) network

| FID Range | Interpretation    |
|-----------|-------------------|
| FID < 10  | Excellent quality |
| 10–50     | Good quality      |
| 50–100    | Moderate quality  |
| > 100     | Poor quality      |

$$FID = ||\mu_r - \mu_g||_2^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Lower FID ↓ → Better image quality & realism ↑

- $\mu_r, \Sigma_r$  are the mean and covariance of real image features.
- $\mu_g, \Sigma_g$  are the mean and covariance of generated image feature
- The first term measures the difference between feature means.
- The second term measures the difference in feature spread and correlations.

## Inception Score (IS)

Inception Score checks whether generated images are clear and meaningful, and whether the generated images are varied from each other, where a higher IS means better image quality and diversity.

$$IS = \exp(\mathbb{E}_x [KL(p(y|x) || p(y))])$$

where:

| Inception Score | Interpretation         |
|-----------------|------------------------|
| $\sim 1.0$      | Random noise           |
| 8.0–10.0        | Good generative models |
| 11.0–12.0       | Real CIFAR-10 images   |

- $p(y|x)$  is the predicted class distribution for an image.
- $p(y)$  is the marginal distribution across all generated images.
- $KL$  is the Kullback–Leibler divergence.

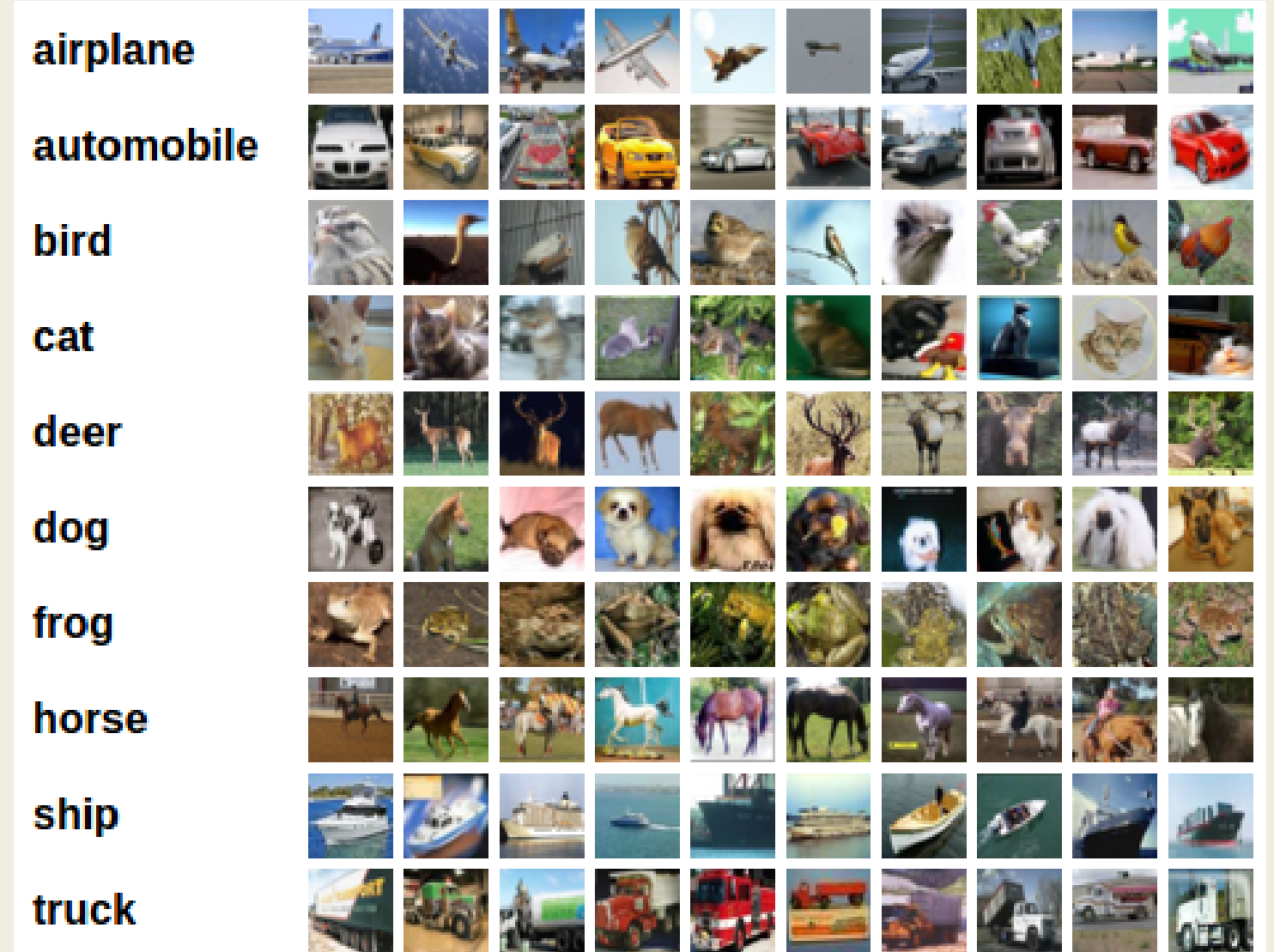
**Higher IS  $\uparrow$   $\rightarrow$  Better image quality & diversity  $\uparrow$**

- Higher IS means:
- Each image has a sharp and confident class prediction (high quality).
- Different images belong to varied classes (high diversity).

# *Overview of the Ablation Study Project and Dataset*

This project investigates score-based generative models, focusing on CIFAR-10 datasets .

## Used CIFAR-10 Dataset



**60,000 images of 10 classes**

**In this section, we focus on the training process of the model.**

**Dataset: CIFAR-10 (32×32) Cats (Class Index = 3)**

- Training Steps: 100,000 , training time ~ 7 hrs per model
- Model Batch Size: Same across all models
- (from Level-1 search) UNet Base Architecture:
- `model.nf = 128`
- `model.ch_mult = (1, 2, 2, 2)`
- `model.num_res_blocks = 1`

| <i>Model Type</i>  | <i>Time</i>       | <i>FID</i>          | <i>Inception Score</i> |
|--------------------|-------------------|---------------------|------------------------|
| VE-DDPM            | 5h 53min          | 36.091696079        | 4.198 ± 0.351          |
| <i>VP - ddpmpp</i> | <i>5h 37m 17s</i> | <i>28.663293108</i> | <i>5.116 ± 0.476</i>   |

VP-ddpmpp achieved the highest Inception Score (5.116), indicating better image quality and diversity.

VP-ddpmpp also had the fastest training time (~5h 37m), making it computationally efficient.

VE-DDPM trained relatively fast but produced a higher FID, meaning lower realism compared to other models.

Overall, different SDE-based models show a tradeoff between training time, image realism (FID), and diversity (IS).

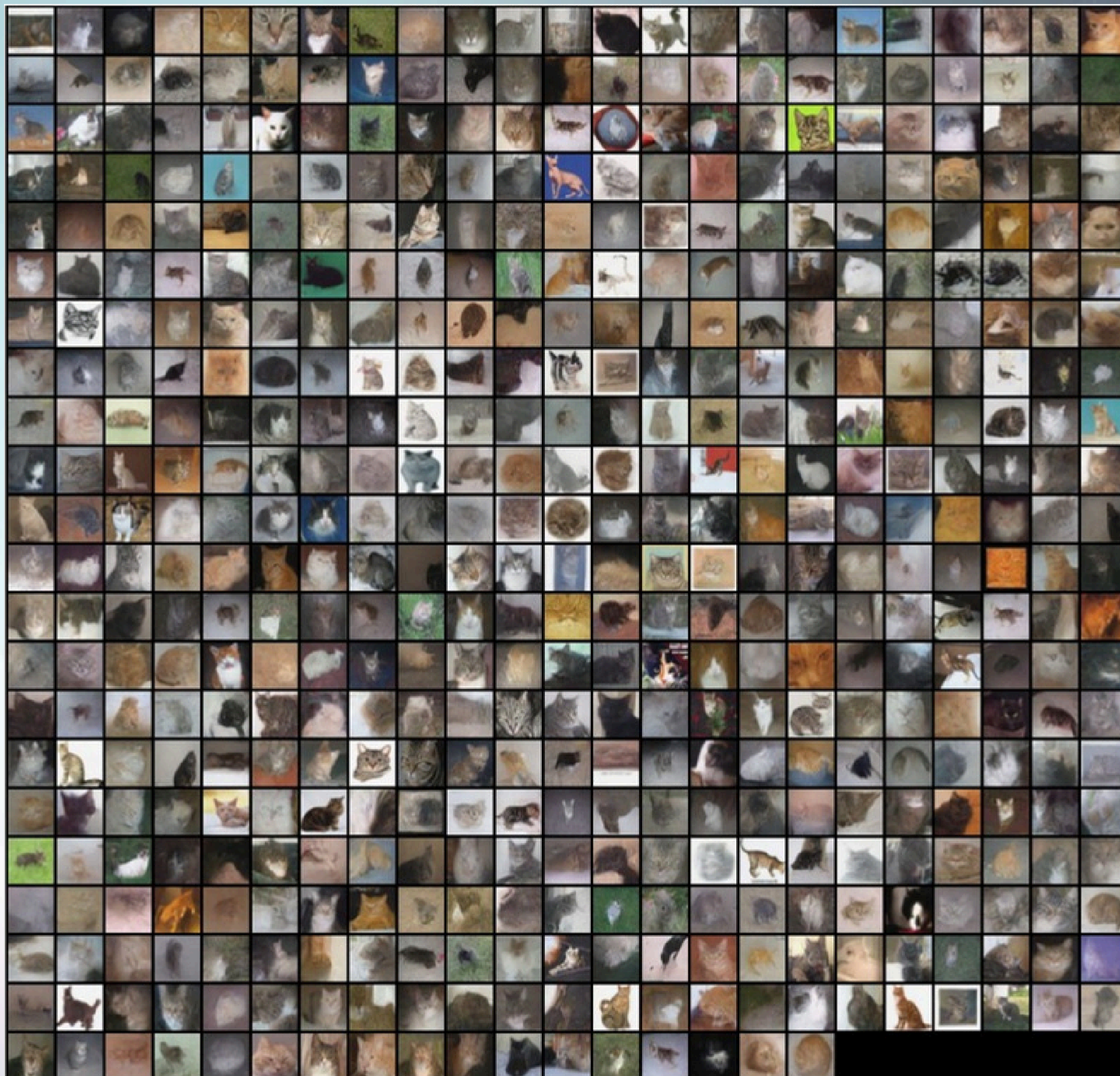
# VP - ddpm++

100000 steps

Time : 5h 37m 17s

FID: 28.663293108

Inception Score:  $5.116 \pm 0.476$

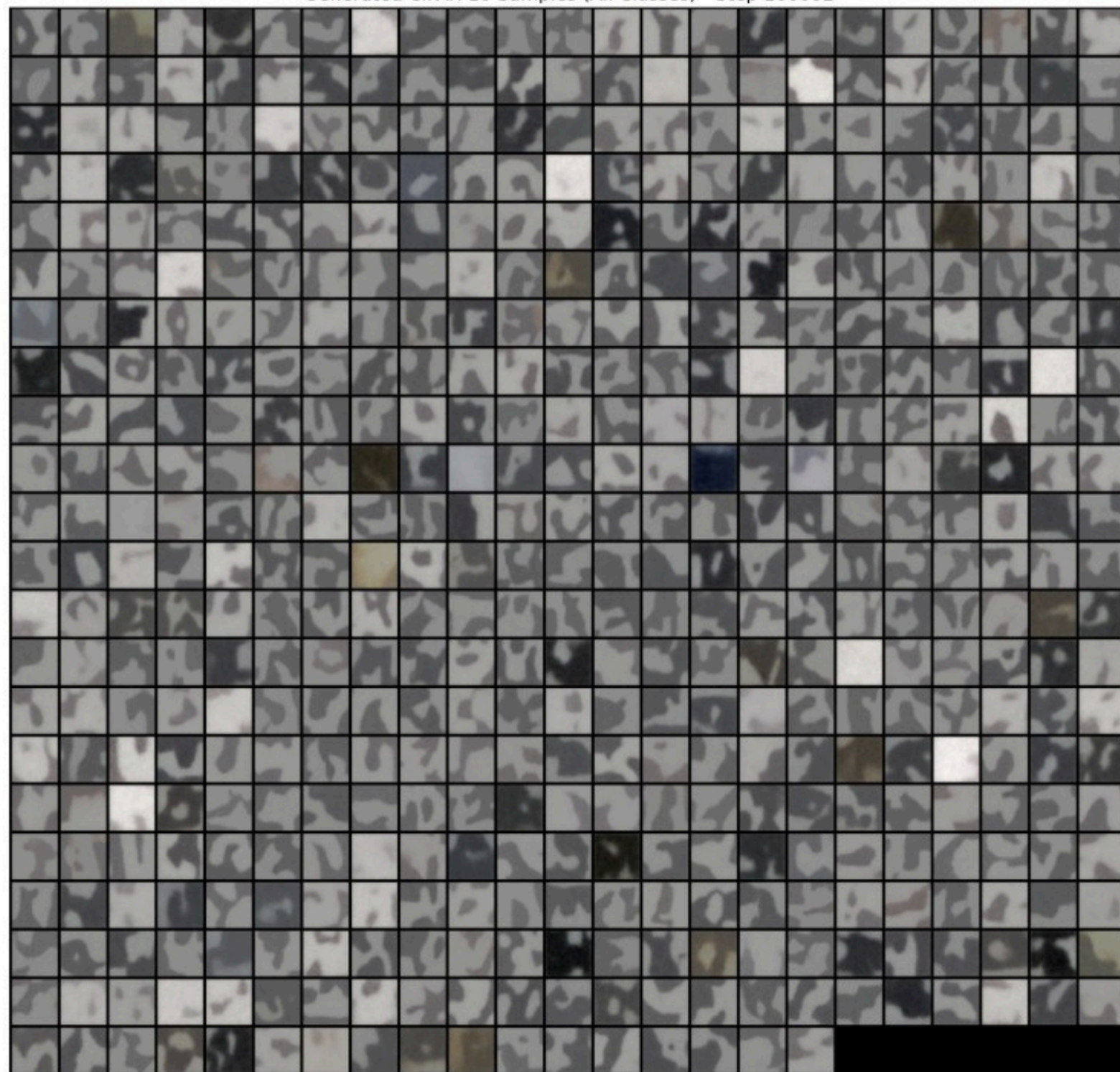


Sampling steps (timesteps): 50  
Generation time: 24.68 seconds  
**FID: 257.07461094320718**

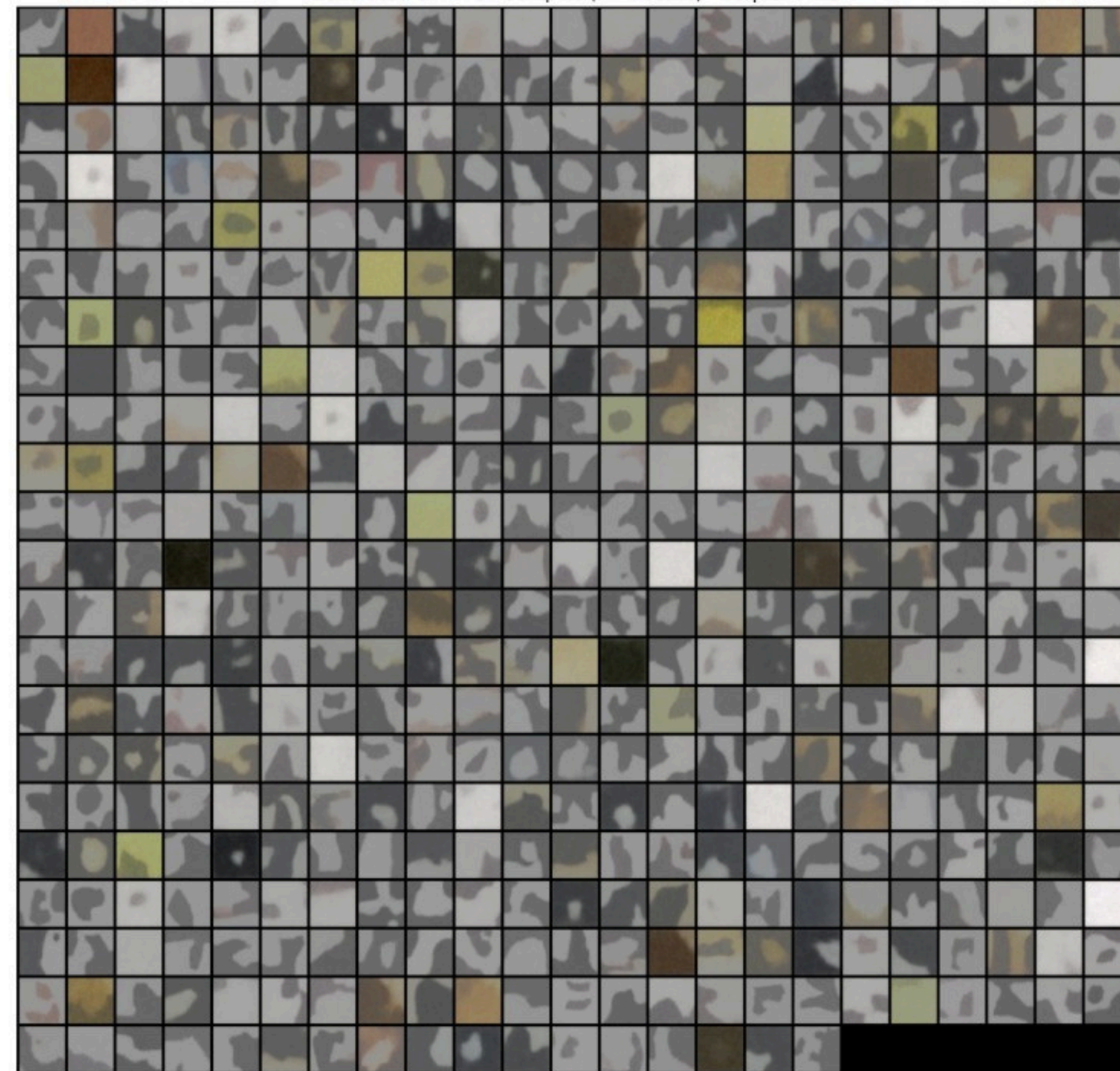
## VP ddpm++

Sampling steps (timesteps): 100  
Generation time: 49.59 seconds  
**FID: 215.2395501632671**

Generated CIFAR-10 Samples (All Classes) - Step 100001



Generated CIFAR-10 Samples (All Classes) - Step 100001



Sampling steps (timesteps): 500

Generation time: 4.13 minutes

FID: 182.72723578137314

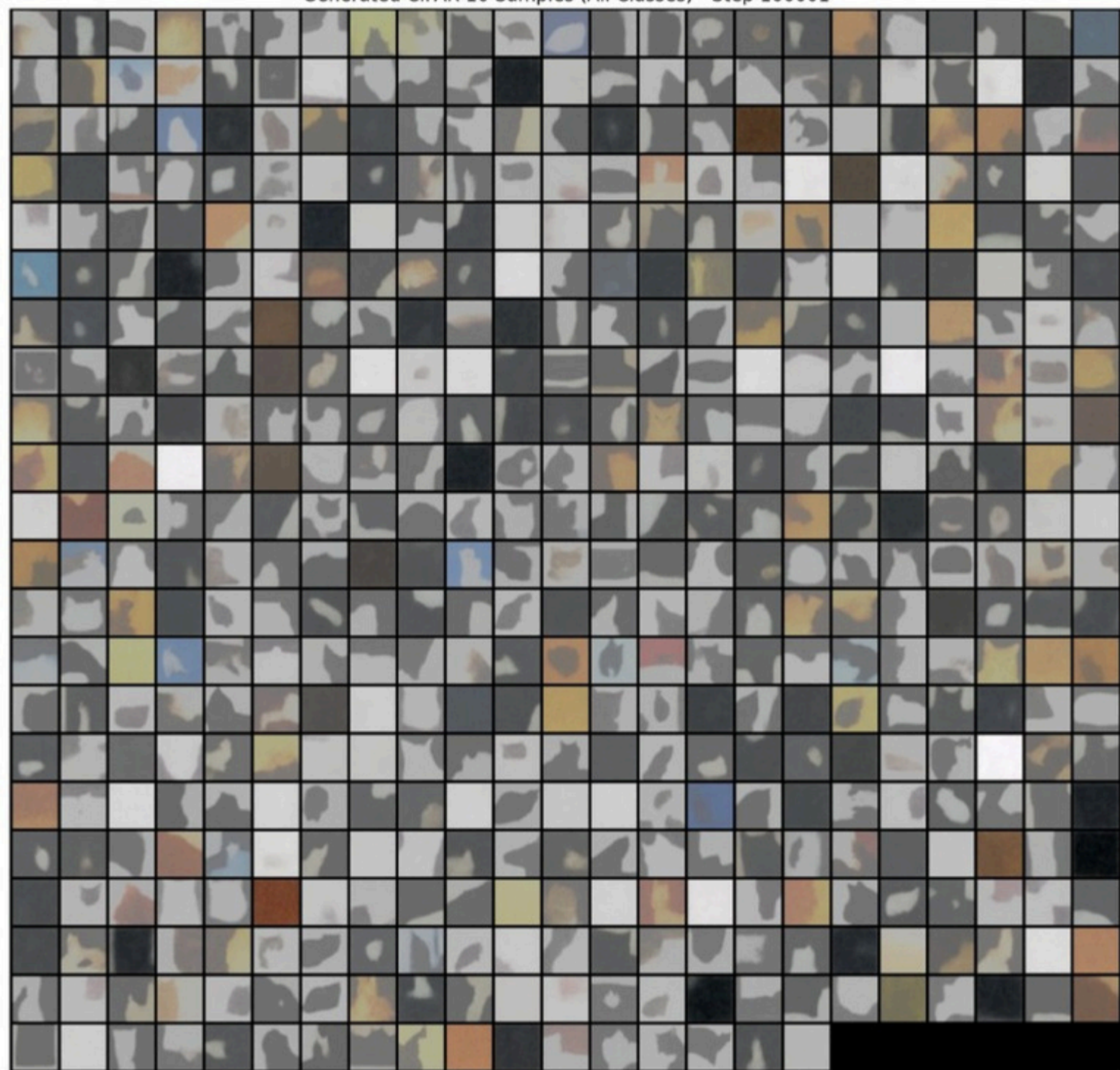
**VP ddpm++**

Sampling steps (timesteps): 1000

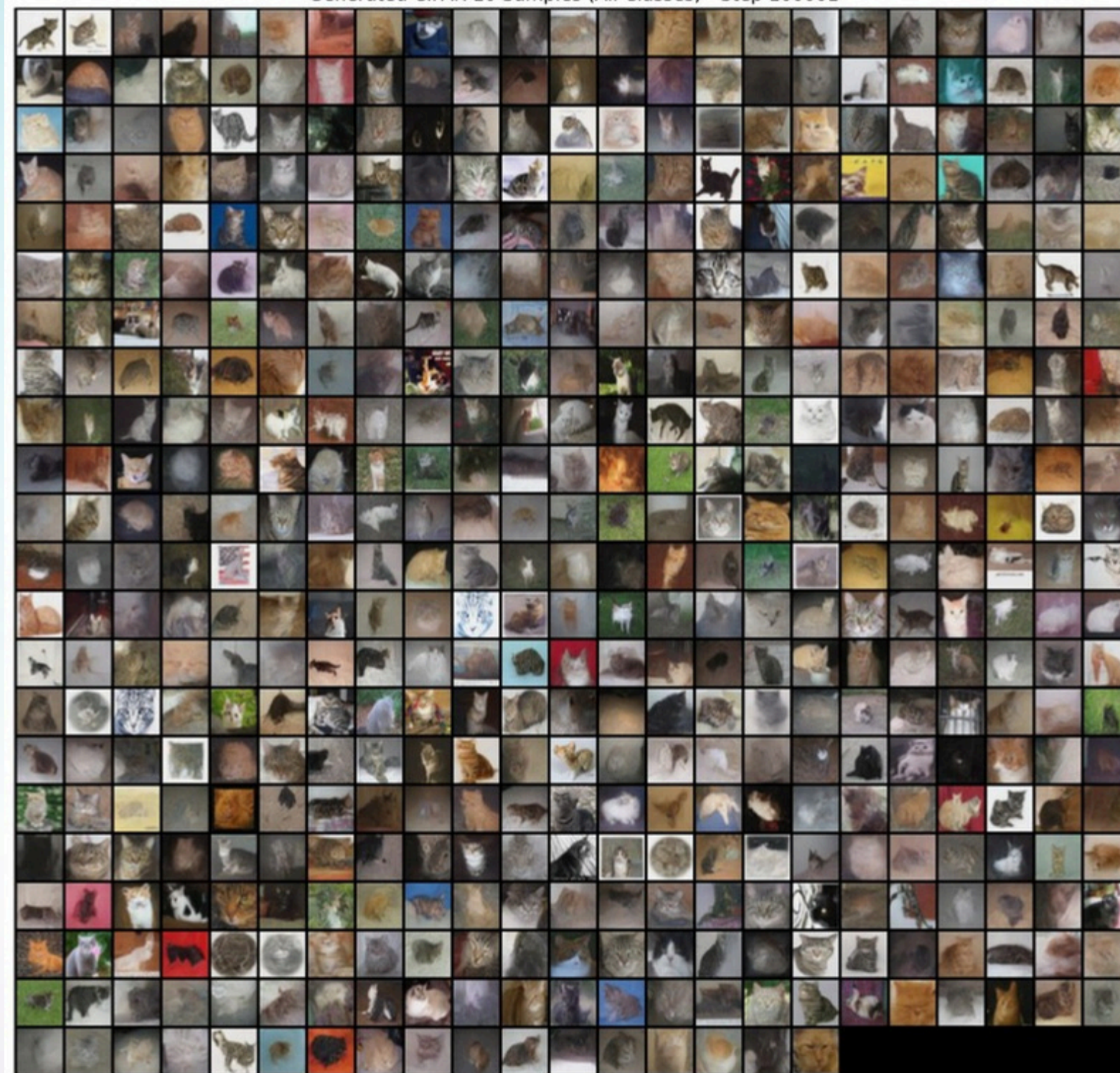
Generation time: 8.23 minutes

FID: 28.663293108

Generated CIFAR-10 Samples (All Classes) - Step 100001



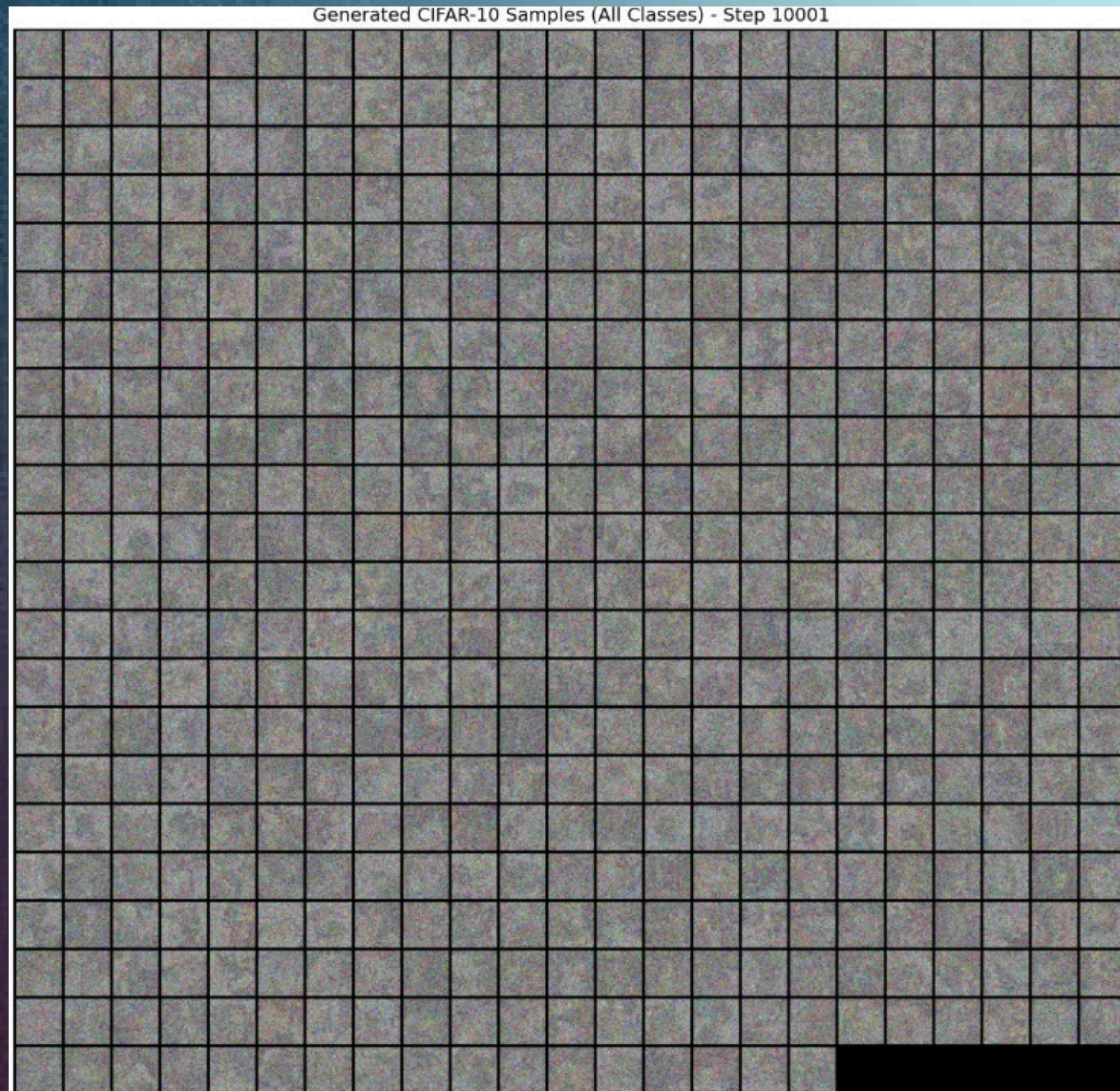
Generated CIFAR-10 Samples (All Classes) - Step 100001



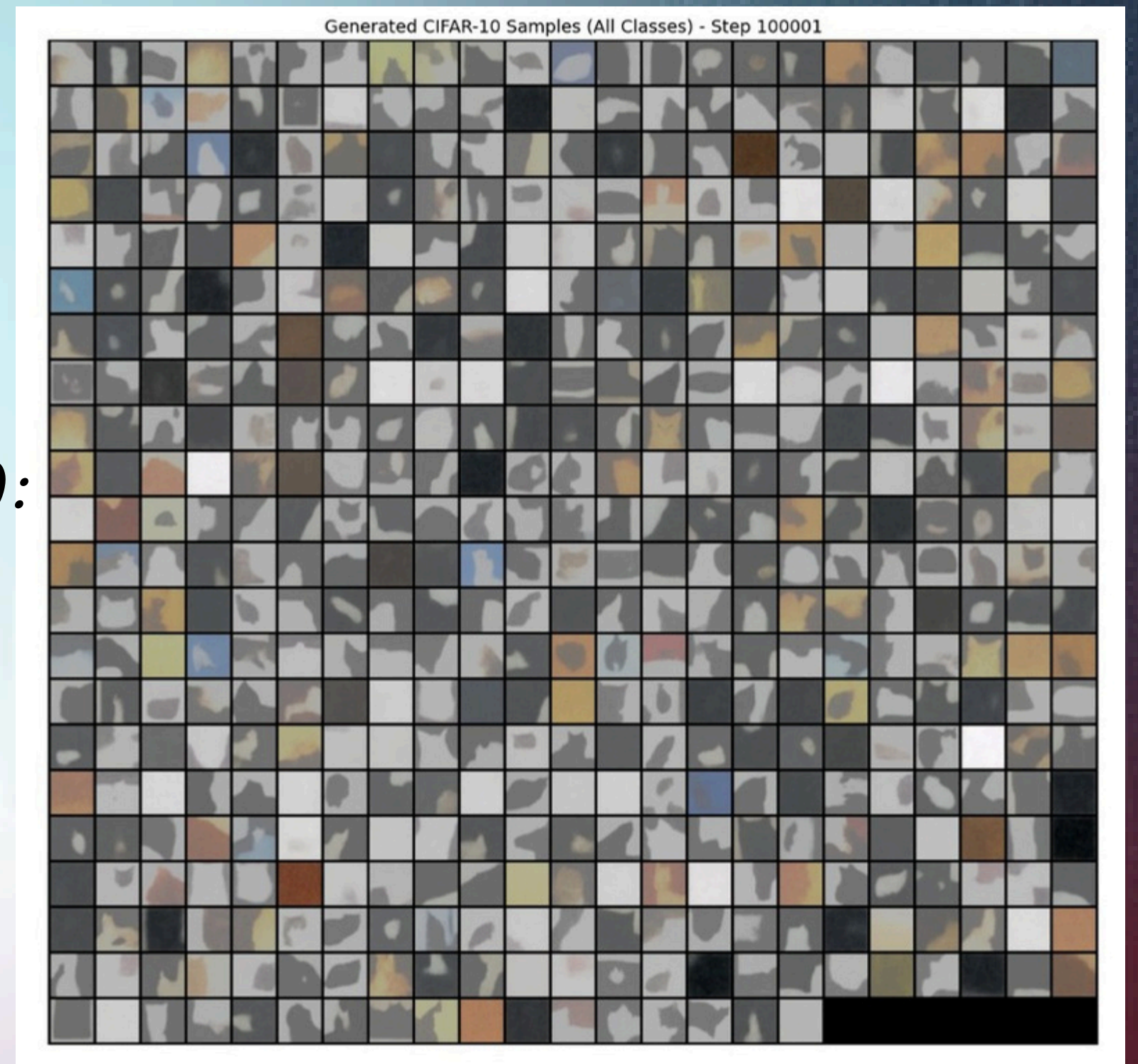
Sampling steps (timesteps): 500

difference btw the intermediate samples atstep - 500

VE(variance exploding)      VP(variance preserving)



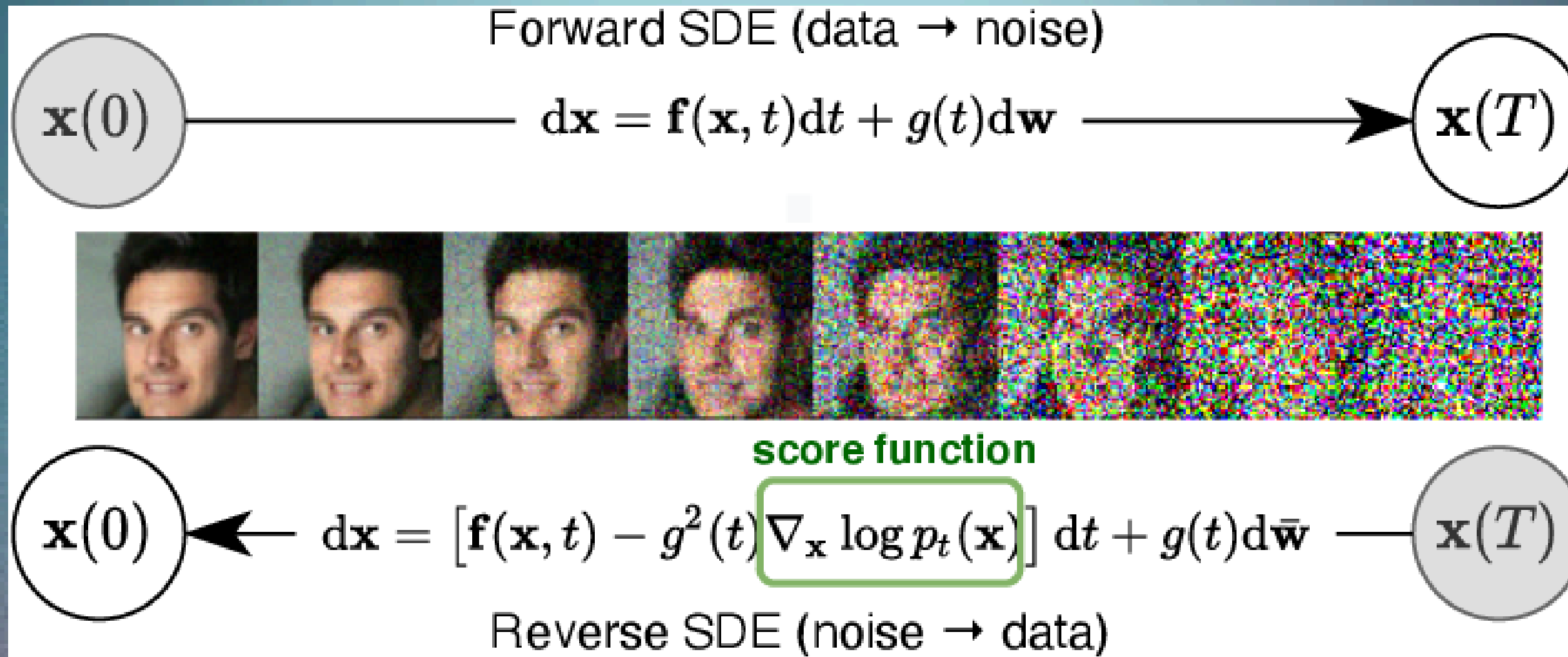
*Sampling  
steps  
(timesteps):  
500*



## **vp\_ddpmpp**

| <i>Sampling steps</i> | <i>Time</i> | <i>FID</i>  |
|-----------------------|-------------|-------------|
| 50                    | 24.68       | 257.0746109 |
| 100                   | 49.59       | 215.2395501 |
| 500                   | 247.95      | 182.728797  |
| 1000                  | 496.19      | 30.75913579 |

## Key Observation:



- Score-based generative models use SDEs to gradually add noise to data and learn how to reverse the process.
- The model learns the score function, which predicts the direction to remove noise from images.
- Reverse-time SDEs transform pure noise into realistic images step by step.
- The stochastic nature of SDEs improves image diversity and generation stability.
- These models achieve high-quality image synthesis with low FID and strong realism.